

## # StockFlow — Inventory & Purchase Management Platform

### ### Full Build Guide & Master Prompt (Project 1 of 3)

> **\*\*How to use this file:\*\*** Paste this entire document into a new Claude.ai chat as your first message. Then work through it section by section — ask Claude to generate one part at a time (e.g., "Let's start with Step 1: backend setup"), copy the code it gives you into your project, and move to the next step. Don't ask for everything in one message; you'll get better, more explainable code by building incrementally.

---

#### ## 1. Project Brief (give this context first)

**\*\*What I'm building:\*\*** StockFlow, a full-stack inventory and purchase management system for a small retail/electronics business. This is a portfolio project to demonstrate real backend engineering skill for junior MERN developer job applications — not a tutorial clone.

**\*\*The real-world problem it solves:\*\*** Small shops track stock in spreadsheets or memory. Stock counts drift from reality because sales, purchases, damages, and returns aren't recorded as discrete events — only the "current quantity" is edited directly. Nobody can explain why the number is what it is, and there's no way to catch errors.

**\*\*The core engineering idea — read this carefully, it drives every schema decision below:\*\***

Do NOT model inventory as `product.quantity` that gets directly edited. Instead:

- Every stock change (purchase received, sale, damage, return) creates an

**\*\*InventoryMovement\*\*** record.

- A product's current stock is the **\*\*sum of its movements\*\***, not a field you overwrite.

- This gives a full audit trail: you can always answer "why is this 47 and not 50?"

**\*\*I am NOT trying to build:\*\*** a generic product CRUD app, a full accounting system, or anything using microservices/Docker/Kubernetes/GraphQL. Keep the stack simple and defensible.

---

## ## 2. Tech Stack

| Layer | Choice | Notes |

| --- | --- | --- |

| Frontend | React (Vite), React Router, Axios | Vite over CRA — faster, modern default |

| Styling | Tailwind CSS | Fast to build clean UI without a design system |

| State | React Context API (+ `useReducer` where needed) | Redux Toolkit is optional — only add if state gets genuinely complex |

| Backend | Node.js + Express.js | REST API |

| Database | MongoDB (local, installed on your machine) + Mongoose | Local for development; switch to MongoDB Atlas free tier only when deploying (Section 12) |

| Auth | JWT (access token) + bcrypt for password hashing | No third-party auth provider — build it yourself, it's an interview topic |

| Validation | express-validator or Zod | Server-side validation is a must-have, not optional |

| File/CSV export | `json2csv` or `papaparse` (backend) | For the "Export Inventory" feature |

| Deployment (backend) | Render (free tier) | Easiest free Node hosting with env var support |

| Deployment (frontend) | Vercel | You already know this from your portfolio |

| Deployment (DB) | MongoDB Atlas free cluster | |

| Version control | Git + GitHub | Commit often, meaningful messages — this is part of your story too |

---

### ## 3. Prerequisites (do this before writing code)

1. Install Node.js LTS (v20+) — verify with ``node -v``
2. Install VS Code (or your preferred editor)
3. Confirm your local MongoDB server is running. On most setups:
  - Windows (installed as a service): it usually starts automatically — check the "MongoDB Server" service in Task Manager, or run ``mongod --version`` to confirm it's installed.
  - macOS/Linux: start it manually with ``mongod`` (or ``brew services start mongodb-community`` on Mac if installed via Homebrew).
  - Verify it's listening by connecting with ``mongosh`` (MongoDB Shell) — if it connects, you're good.
  - Your local connection string will be: ``mongodb://127.0.0.1:27017/stockflow`` (Mongoose creates the ``stockflow`` database automatically on first write — you don't need to create it manually).
4. **\*\*Later, at deployment time only (Section 12), you'll need a free MongoDB Atlas cluster\*\***  
— your deployed backend can't reach your local machine's database, so production will point to Atlas while your local dev stays on local MongoDB. No code changes needed for this, just a different ``MONGO_URI`` value.
5. Create a GitHub account/repo for this project (``stockflow``)
6. Install Postman or Thunder Client (VS Code extension) for testing API routes before building the frontend

---

### ## 4. Folder Structure

...

stockflow/

```
└─ backend/  
  │ └─ src/  
    │ │ └─ config/  
    │ │ │ └─ db.js  
    │ │ └─ models/  
    │ │ │ └─ User.js  
    │ │ │ └─ Product.js  
    │ │ │ └─ Category.js  
    │ │ │ └─ Supplier.js  
    │ │ │ └─ PurchaseOrder.js  
    │ │ │ └─ Sale.js  
    │ │ │ └─ InventoryMovement.js  
    │ │ │ └─ AuditLog.js  
    │ │ └─ controllers/  
    │ │ │ └─ authController.js  
    │ │ │ └─ productController.js  
    │ │ │ └─ supplierController.js  
    │ │ │ └─ purchaseOrderController.js  
    │ │ │ └─ saleController.js  
    │ │ │ └─ inventoryController.js  
    │ │ │ └─ dashboardController.js  
    │ │ └─ routes/  
    │ │ │ └─ authRoutes.js  
    │ │ │ └─ productRoutes.js  
    │ │ │ └─ supplierRoutes.js
```

```
| | | └─ purchaseOrderRoutes.js
| | | └─ saleRoutes.js
| | | └─ inventoryRoutes.js
| | | └─ dashboardRoutes.js
| | └─ middleware/
| | | └─ authMiddleware.js    // verifies JWT
| | | └─ roleMiddleware.js    // RBAC check
| | | └─ errorMiddleware.js   // centralized error handler
| | | └─ validateMiddleware.js
| | └─ utils/
| | | └─ generateToken.js
| | | └─ csvExporter.js
| | └─ app.js
| └─ server.js
| └─ .env.example
| └─ .gitignore
| └─ package.json
└─ frontend/
  | └─ src/
  | | └─ api/
  | | | └─ axiosInstance.js
  | | └─ context/
  | | | └─ AuthContext.jsx
  | | └─ components/
  | | | └─ layout/ (Navbar, Sidebar, ProtectedRoute)
```

```
| | | └─ products/
| | | └─ purchases/
| | | └─ inventory/
| | | └─ dashboard/
| | └─ pages/
| | └─ Login.jsx
| | └─ Dashboard.jsx
| | └─ Products.jsx
| | └─ ProductDetail.jsx
| | └─ Suppliers.jsx
| | └─ PurchaseOrders.jsx
| | └─ Sales.jsx
| | └─ InventoryHistory.jsx
| └─ App.jsx
| └─ main.jsx
└─ .env.example
└─ tailwind.config.js
└─ package.json
└─ README.md
...

---
```

## ## 5. Backend Packages to Install

```
```bash
# In /backend

npm init -y

npm install express mongoose dotenv cors bcryptjs jsonwebtoken express-validator morgan

npm install -D nodemon
```
```

## ## 6. Frontend Packages to Install

```
```bash
# In /frontend (after npm create vite@latest frontend -- --template react)

npm install axios react-router-dom

npm install -D tailwindcss postcss autoprefixer

npx tailwindcss init -p
```
```

---

## ## 7. Database Schema (Mongoose Models)

Give Claude this exact field structure when generating models — don't let it improvise the core fields:

**\*\*User\*\***

```

name, email (unique), password (hashed), role: enum['admin','staff'], createdAt

...

### **\*\*Category\*\***

...

name, description

...

### **\*\*Supplier\*\***

...

name, contactPerson, email, phone, address, createdAt

...

### **\*\*Product\*\***

...

name, sku (unique), category (ref Category), supplier (ref Supplier),

unitPrice, costPrice, minStockLevel, isActive, createdAt

// NOTE: no "quantity" field here — current stock is CALCULATED from InventoryMovement

...

### **\*\*InventoryMovement\*\*** (the core of the whole project)

...

product (ref Product), type: enum['purchase','sale','damage','return','adjustment'],

quantity (positive or negative), reason, reference (e.g. PO number or Sale ID),

performedBy (ref User), createdAt

...



### **\*\*PurchaseOrder\*\***

...

poNumber (unique, auto-generated), supplier (ref Supplier),  
items: [{ product (ref Product), quantity, unitCost }],  
status: enum['draft','pending','approved','received','cancelled'],  
createdBy (ref User), approvedBy (ref User), receivedAt, createdAt  
// BUSINESS RULE: inventory only increases when status changes to 'received' —  
// this must be enforced in the controller, not just the schema.

...

### **\*\*Sale\*\***

...

saleNumber (unique), items: [{ product (ref Product), quantity, unitPrice }],  
totalAmount, soldBy (ref User), createdAt  
// Creating a Sale should auto-create matching InventoryMovement records (type: 'sale',  
negative qty)

...

### **\*\*AuditLog\*\***

...

action, entityType, entityId, performedBy (ref User), details (text), createdAt

...

---

## **## 8. API Routes (build and test each with Postman before touching the frontend)**

...

## AUTH

POST /api/auth/register (admin only, or open initially for setup)

POST /api/auth/login

GET /api/auth/me (protected)

## PRODUCTS

GET /api/products ?page=&limit=&category=&search=&stockStatus=

POST /api/products (admin)

GET /api/products/:id

PUT /api/products/:id (admin)

DELETE /api/products/:id (admin)

GET /api/products/:id/stock → calculated current stock + recent movements

## SUPPLIERS

GET/POST/PUT/DELETE /api/suppliers (admin)

## PURCHASE ORDERS

GET /api/purchase-orders ?status=&supplier=

POST /api/purchase-orders (create as 'draft')

PUT /api/purchase-orders/:id/status (draft→pending→approved→received)

// THIS is where inventory movements get created, only on 'received'

## SALES

GET /api/sales

POST /api/sales // auto-creates InventoryMovement (type: sale)

## INVENTORY

GET /api/inventory/movements ?product=&type=&dateFrom=&dateTo= (the audit trail view)

GET /api/inventory/low-stock → products where current stock <= minStockLevel

GET /api/inventory/export → CSV download

## DASHBOARD

GET /api/dashboard/summary → totals: products, low stock, pending POs, sales today

...

---

## ## 9. Build Order (follow this sequence — don't jump ahead)

1. **Backend skeleton**: `server.js`, `app.js`, MongoDB connection, health-check route.

Confirm it runs.

2. **User model + auth**: register, login, JWT generation, `authMiddleware`, `roleMiddleware`.

Test in Postman.

3. **Category & Supplier**: simplest CRUD, good warm-up before the harder models.

4. **Product model + CRUD**: including pagination, search, and category filter on GET.

5. **InventoryMovement model + calculated stock**: write the helper function that sums movements for a product — this is the most important piece of logic in the whole project. Test it with manually-inserted movement documents before wiring it to purchases/sales.

6. **PurchaseOrder model + status workflow**: build the state machine (draft→pending→approved→received) and enforce that InventoryMovement is only created on the transition to `received`.

7. **\*\*Sale model\*\***: creating a sale should validate enough stock exists, then create the Sale + matching InventoryMovement in the same operation.
8. **\*\*Low-stock + dashboard endpoints\*\***: aggregate queries.
9. **\*\*AuditLog\*\***: hook into key actions (status changes, stock adjustments).
10. **\*\*CSV export\*\***.
11. **\*\*Frontend\*\***: auth pages → protected routing → product list/detail → purchase order flow → sales flow → inventory history table → dashboard charts (use `recharts`).
12. **\*\*Polish\*\***: loading/error/empty states everywhere, form validation feedback, responsive layout.
13. **\*\*Deployment\*\***.

---

## ## 10. Environment Variables

**\*\*backend/.env\*\*** (development — local MongoDB)

...

PORT=5000

MONGO\_URI=mongodb://127.0.0.1:27017/stockflow

JWT\_SECRET=your\_random\_secret\_string

JWT\_EXPIRES\_IN=7d

CLIENT\_URL=http://localhost:5173

...

> When you deploy (Section 12), you'll create a free MongoDB Atlas cluster and replace `MONGO\_URI` with the Atlas connection string in Render's environment variables — your local `.env` file stays pointed at local MongoDB and is never affected.

**\*\*frontend/.env\*\***

...

VITE\_API\_URL=http://localhost:5000/api

...

Remember: never commit `.env` — only commit `.env.example` with blank/placeholder values, and confirm `.gitignore` includes `.env` and `node_modules`.

---

## **## 11. Things to Be Able to Explain in an Interview (build with these in mind)**

- Why is current stock calculated from movements instead of stored as a field?
- Why does a Purchase Order only affect inventory when it's `received`, not when it's created?
- How do you prevent a Sale from being created if stock is insufficient?
- How does your JWT auth flow work end to end (login → token → protected route)?
- How does role-based access control differ for admin vs. staff in your routes?
- Why did you add pagination and server-side filtering instead of returning everything and filtering in React?
- What happens if two requests try to update the same product's stock at nearly the same time? (You don't need a perfect solution — but you should know where the risk is and what MongoDB feature could help, e.g. atomic `findOneAndUpdate` operations or transactions.)

---

## **## 12. Deployment Checklist**

1. Push backend and frontend to separate GitHub repos (or a monorepo with two folders — either is fine, mention your choice in the README).
2. **\*\*Create your production database\*\***: sign up for MongoDB Atlas (free), create a free M0 cluster, add a database user, allow network access (0.0.0.0/0 for simplicity, or Render's IPs if you want to be stricter), and copy the connection string — this replaces your local `MONGO\_URI` only in production.
3. **\*\*Backend → Render\*\***: connect repo, set build command `npm install`, start command `node server.js`, add all env vars in Render's dashboard — using the **\*\*Atlas\*\*** connection string here, not your local one.
4. **\*\*Frontend → Vercel\*\***: connect repo, set root directory to `frontend`, add `VITE\_API\_URL` pointing to your deployed Render backend URL.
5. Update backend's `CLIENT\_URL` / CORS config to allow your deployed Vercel domain.
6. Test the full deployed flow: register/login, create a product, create+receive a purchase order, confirm stock updates, create a sale, confirm stock decreases, check inventory history.
7. Write the README: problem statement, tech stack, screenshots, setup instructions, live demo link, and 3–4 bullet points describing the core engineering decisions (movement-based inventory, PO state machine, RBAC).

---

### ## 13. MVP vs. Stretch Goals

#### **\*\*MVP (build this first, fully working end to end):\*\***

Auth + RBAC, Product/Supplier/Category CRUD, PurchaseOrder workflow with movement creation on `received`, Sale creation with stock validation, low-stock dashboard, inventory movement history view.

#### **\*\*Stretch (only after MVP is fully deployed and working):\*\***

CSV export, sales charts over time, audit log UI, product images, multi-warehouse support (only if you want to go further — not required).

---

*\*End of StockFlow build guide. Work through sections 9 (Build Order) one numbered step at a time in your new chat.\**